UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI MATEMATICA

LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI PROGRAMMAZIONE AD OGGETTI

Progettazione di un applicativo per E-commerce

Zakaria Laoud - 2113196

Indice

1 Introduzione **1**

2 Descrizione del modello **1**

3 Polimorfismo **3**

4 Persistenza dei dati **4**

5 Funzionalità implementate **4**

6 Rendicontazione ore **6**

1 Introduzione

E-commerce è un'applicazione per la gestione dei prodotti in vendita ispirata, per alcuni aspetti, ad Amazon, E-bay e Zalando. Essa è composta da un'interfaccia grafica totalmente scritta in Qt, mentre la parte logica è in C++.

Tra i prodotti in vendita è possibile creare, cancellare, modificare e visualizzare articoli appartenenti alle macro-categorie: abbigliamento, elettronica e media stampati (es. libri, articoli di giornale, fumetti ecc...).

L'applicazione è perlopiù incentrata sul dare all'utente un'esperienza fluida, permettendogli di *personalizzare la visualizzazione* dei prodotti e costruire una propria *lista dei preferiti*.

Inoltre è previsto un sistema di ricerca basato sulle caratteristiche generali di un prodotto e, scelta una determinata categoria, anche su alcuni attributi caratteristici della stessa.

L'obiettivo fin da subito è stato quello di creare una gerarchia di classi che fosse facilmente espandibile, infatti si potrebbero aggiungere altre macro-categorie (es. prodotti per la casa, giardinaggio, pezzi meccanici, ricambi per auto/bici ecc...) rendendo l'applicazione il più completa possibile.

2 Descrizione del modello

Il modello logico su cui si basa l'applicazione è formato da una classe base astratta *Product* che racchiude le principali caratteristiche che accomunano tutti i prodotti in vendita: nome, descrizione, prezzo, immagine di copertina e un booleano che indica se, per l'utente, l'articolo è o meno uno dei suoi preferiti.

Al di sotto di tale classe troviamo tre sottoclassi concrete:

- *Clothing* che rappresenta i prodotti appartenenti alla macro-categoria abbigliamento, caratterizzato da una taglia, una serie di materiali e il genere a cui è destinato.

- *Electronics* che rappresenta gli apparecchi elettronici caratterizzati da un marchio, una copertura di garanzia espressa in mesi e la tipologia di alimentazione di cui necessitano.

- *PrintedMedia* che rappresenta tutto l'insieme dei prodotti stampati, come libri, megazine, giornali ecc. E' caratterizzato dalla tipologia di media che rappresenta, l'autore oppure editore responsabile della sua stesura, la casa editrice responsabile della sua pubblicazione e il numero di pagine di cui è composto.

Inoltre, dato che l'*operator==* è necessario al corretto funzionamento della logica, è stato predisposto il suo override all'interno di ogni classe.

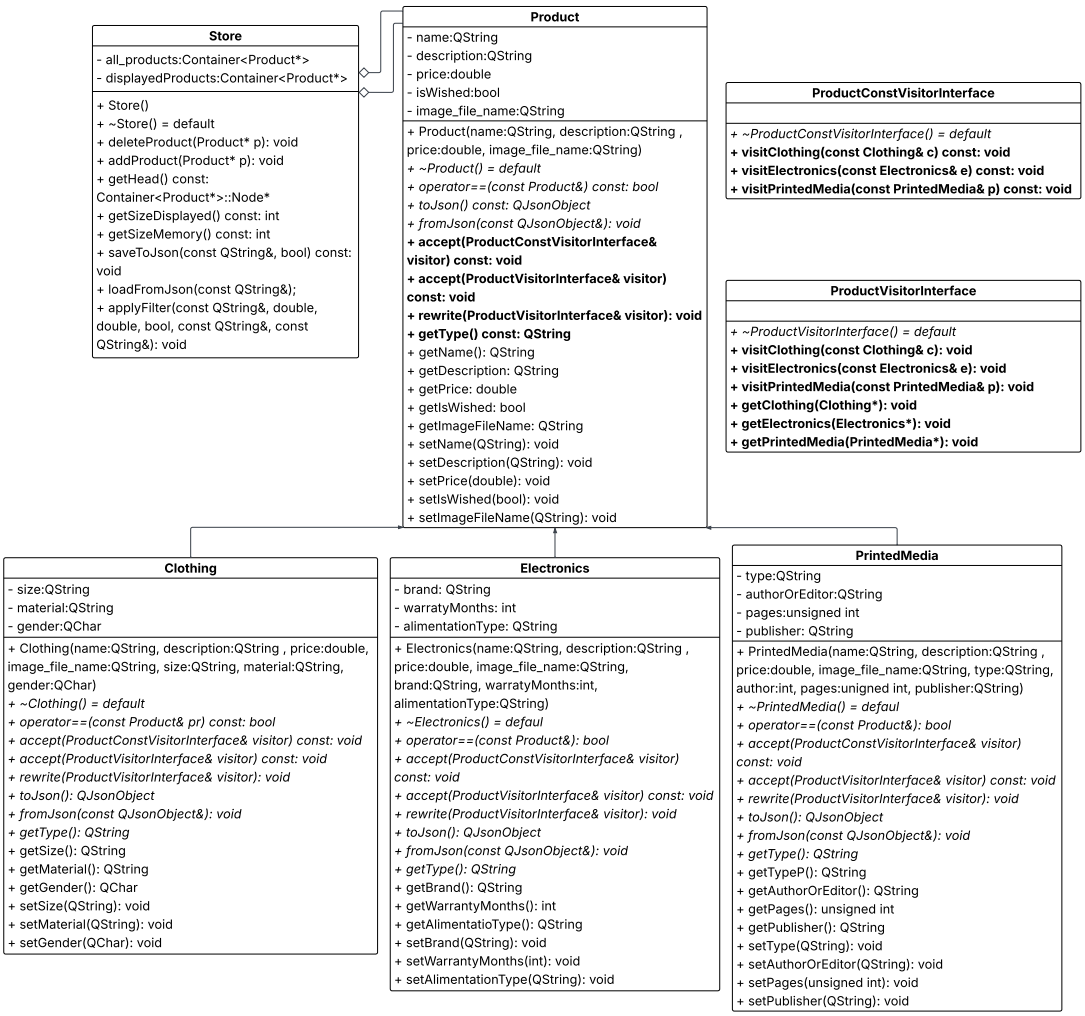

Figura 1: Schema logico

Per il salvataggio e caricamento dei prodotti in formato Json sono stati previsti i metodi *toJson()* e *fromJson()*, già nella classe base astratta Product, i quali si appoggiano allo strumento *QJsonObject* di Qt per la de-serializzazione e serializzazione dei prodotti.

E' importante tenere conto che il metodo virtuale *getType()* all'interno della classe Product è usato semplicemente per indicare, durante la serializzazione di un oggetto, a che sottotipo di Product appartiene, quindi in nessun caso per gestire il controllo del flusso.

Un ulteriore elemento chiave è l'uso del Visitor pattern, che separa il modello dei dati dalla logica di visualizzazione e modifica, riducendo l'accoppiamento e semplificando l'estensibilità. Sono state create due interfacce differenti *ProductVisitorInterface* e *ProductConstVisitorInterface* che differiscono per il fatto che la seconda non modifica l'oggetto d'istanza, permettendo di rispettare la *const correctness* del C++.

Nell'interfaccia *ProductConstVisitorInterface* sono stati inseriti altri tre metodi get, uno per tipo concreto. Questi metodi permettono di aggiornare i campi caratteristici di ogni classe concreta tramite il richiamo del metodo virtuale puro *rewrite()*, definito nella classe base Product. In questo modo, ogni visitor può aggiornare in modo sicuro i dati specifici di un prodotto senza violare la costanza dell'oggetto quando necessario. Questo approccio è un tentativo di seguire la logica dei visitor pattern, i quali sfruttano il metodo *accept()*. Ovviamente nella classe Product sono inseriti due metodi accept, uno per tipo di visitor.

Una volta inserito un prodotto, per garantire una corretta visualizzazione dell'immagine di copertina, è stata inserita una classe di servizio **FileUtils**, costituita solamente da metodi *statici*:

- *getProjectPath()* che recupera la directory della cartella del progetto (contenente quindi le sottocartelle *Images* e *Examples*).

- *moveToImages()* che permette di spostare le immagini selezionate dall'utente all'interno della cartella *Images* del progetto, permettendo di ritrovarle indipendentemente dal dispositivo in cui viene utilizzata l'applicazione. Altri vantaggi sono che bisogna solo memorizzare il nome dell'immagine, in quanto già dentro Images, e si evita l'utilizzo di percorsi assoluti

Adibita alla gestione di tali prodotti c'è la classe *Store*, che racchiude al suo interno due vettori (*liste* implementate come template all'interno della classe di servizio **Container**) tra le quali: *all_products* contiene i puntatori di tutti i prodotti in memoria, mentre *displayedProducts* contiene i puntatori ai prodotti che rispettano i filtri inseriti dall'utente.

Per quanto riguarda l'applicazione dei filtri, il metodo *applyFilter*, scorrendo *all_products*, verifica per ciascuno se soddisfa i criteri specificati (contiene una determinata parola all'interno del nome prodotto oppure dentro alla descrizione, prezzo massimo, prezzo minimo, la macro-categoria e alcuni campi extra caratteristici) e, in caso affermativo, lo aggiunge a *displayedProducts*, altrimenti continua fino alla fine della lista.

L'uso di puntatori consente di mantenere una singola istanza di ogni prodotto in memoria, evitando duplicazioni e rendendo più semplice la propagazione delle modifiche a tutte le viste in cui il prodotto compare. Ovviamente la classe *Container* è stata costruita in modo tale che se svuoto *displayedProducts*, vada semplicemente a distruggere i nodi, lasciando i Products intatti, in quanto ancora puntati da *all_products* (i prodotti stessi verranno eliminati solamente se procedo a cancellare *all_products*).

3 Polimorfismo

La principale forma di polimorfismo utilizzata all'interno dell'applicazione riguarda il *design pattern visitor*, sfruttato per la creazione delle interfacce di visualizzazione (SingleProduct e Miniature) e modifica (SingleProductEdit).

Più nello specifico:

- *ProductEditorVisitor*, che eredita da ProductVisitorInterface, permette, mediante il metodo *rewrite()*, la sovrascrittura dei campi extra di ogni classe concreta.
- *ProductIconVisitor* che eredita da ProductConstVisitorInterface, permette, con il classico metodo *accept()*, di inserire sulle miniature dei prodotti un'icona caratteristica della macro-categoria a cui appartengono i prodotti stessi.
- *ProductSingleProductVisitor* che eredita da ProductConstVisitorInterface, permette la visualizzazione dei campi extra.

4 Persistenza dei dati

La persistenza dei dati è gestita tramite il formato JSON, in cui la serializzazione di ciascuna classe astratta è garantita dall'attributo *type*, ed è eseguita dai metodi *loadFromJson()* e *saveToJson()* nella classe *Store*. Un esempio della struttura adottata è disponibile nel file *ex.json*, situato nella cartella Examples del progetto, che contiene tre prodotti distinti per ogni macro-categoria.

5 Funzionalità implementate

Le funzionalità implementate possono essere divise in funzionali e grafiche. Quelle funzionali comprendono:

- gestione della lista dei preferiti dell'utente
- salvataggio diretto nella cartella *Examples* del progetto
- salvataggio in una cartella a scelta libera dell'utente con la possibilità di dare un nome specifico al file .json
- salvataggio dei risultati di una ricerca oppure di tutti i prodotti generati (QDialog durante il salvataggio)
- filtro in base ai campi comuni e alcuni campi extra ritenuti pertinenti nelle ricerche
- utilizzo di un file *resources.qrc* dove sono inserite le icone in modo che l'eseguibile sia sempre in grado di trovarle
- controllo dei dati inseriti, con l'elenco dei campi mancanti (mostrati su QDialog)
- durante la creazione/modifica di un prodotto, l'immagine utilizzata viene copiata nella cartella Images del progetto

Per quanto riguarda le funzionalità grafiche:

- toolbar mobile
- utilizzo di tooltip per migliorare l'esperienza
- utilizzo di icone stilizzate significative sulla toolbar e varie schermate
- label in basso a sinistra per mantenere sempre visibile il numero di prodotti mostrati
- in alto a destra è possibile modificate la visualizzazione della schermata principale (grid o vertical)
- per la visualizzazione in formato grid è possibile modificare la quantità di prodotti da mettere su ogni riga
- schermate di caricamento per un esperienza più fluida durante la ricerca degli elementi da mostrare
- gestione del ridimensionamento mediante dimensioni minime oppure QScrollBar
- ogni prodotto ha un icona che rappresenta la macro-categoria a cui appartiene
- possibilità di navigare tra schermate senza continui pop-up
- utilizzo di immagini per ogni prodotto (opzionale)
- se un prodotto è sprovvisto di immagine viene utilizzata un immagine di default
- utilizzo di colori e stili (prezzo stile Amazon)
- effetti grafici con il passaggio del mouse sopra le miniature dei prodotti o dei tasti
- utilizzo dell'icona per la lista dei preferiti che cambia quando cliccata
- vengono applicati i filtri: cambiando i parametri di prezzo, cliccando *invio* mentre si sta descrivendo cosa si sta cercando sulla barra di ricerca oppure cliccando il tasto *search*
- tasto *reset* per resettare i filtri inseriti
- utilizzo del logo dell'applicazione, ricorrente in tutti i QDialog utilizzati (questo grazie alla classe *DialogMessage* con un solo metodo astratto *message()*)

6 Rendicontazione ore

Attività	**Ore previste**	**Ore effettive**
Progettazione	5	4
Sviluppo codice modello logico	10	11
Studio del framework Qt	10	8
Sviluppo della GUI	10	14
Test e debug	5	6
Stesura relazione	5	5.5
Totale	**45**	**49**

Tabella 1: Ore di sviluppo

Si nota come alcune attività, in particolare lo sviluppo della GUI, del modello logico e le fasi di debug, abbiano richiesto più tempo del previsto, mentre altre sono state completate più velocemente.

Nel complesso, le ore effettive superano leggermente quelle pianificate. L'aumento è dovuto soprattutto alla costruzione dell'interfaccia grafica, spesso soggetta a cambiamenti radicali e al rifacimento di interi widget.